PROSESSED

Engineering Hiring | Take-Home Assignment

Flutter Developer Intern

Technical Assignment: “ShopSphere” Mobile App

Version 1.0 · Issued 16 October 2026 · Confidential

Company	Prosessed
Role	Flutter Developer Intern
Issued on	Friday, 16 October 2026
Submission deadline	**Wednesday, 21 October 2026, 11:59 PM IST**
Suggested effort	12–16 focused hours (spread across the window as you like)
Backend	DummyJSON public REST API (https://dummyjson.com) — no backend work needed
Deliverables	GitHub repository link · Google Drive link · Android release APK

Dear Candidate,
Thank you for your interest in joining Prosessed. This assignment is designed to reflect the kind of work our mobile team does every day. We are not looking for the biggest feature list; we are looking for **thoughtful UI/UX, clean and testable code, sound architecture, and honest engineering practices**. A smaller app that is well-built will always score higher than a larger app that is fragile.

Contents

1. Objective & what we evaluate
2. The app: ShopSphere
3. Functional requirements
4. API reference (verified endpoints)
5. Technical requirements
6. Git & README guidelines
7. Deliverables & submission
8. Evaluation rubric
9. Rules, policies & FAQ
10. Pre-submission checklist

1. Objective

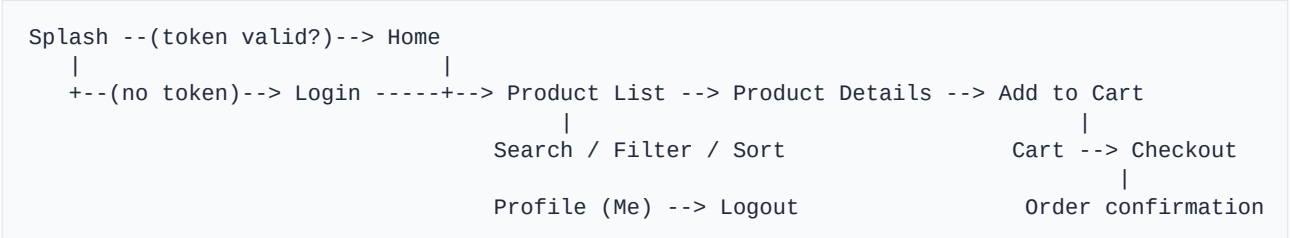
Build a production-quality **Flutter** e-commerce catalogue app, **ShopSphere**, that consumes the DummyJSON REST API. The app lets a user sign in, browse and search products, view product details, manage a cart, and view their profile. Through this single project we will assess the following areas:

Area	What we look for
UI / UX design	Visual polish, consistent theming (Material 3), spacing & typography, intuitive flows, meaningful loading / empty / error states, micro-interactions, accessibility.
State management	Predictable, well-scoped state using one solution (BLoC/Cubit, Riverpod or Provider). No business logic inside widgets.
Clean architecture	Clear separation of presentation, domain and data layers; dependency inversion; dependency injection; feature-first, readable folder structure.
API integration & error handling	Typed models, a dedicated network layer, auth token handling, timeouts, mapping of exceptions to user-friendly failures, retry flows.
Unit testing	Meaningful tests for repositories, use cases and state classes with mocked dependencies.
Git practices	Small, atomic commits with clear messages that tell the story of how the app was built.
Responsive layouts	Layouts that adapt gracefully across phones, tablets, portrait and landscape.

2. The App: ShopSphere

ShopSphere is a lightweight shopping catalogue. You are free to choose your own visual identity (colours, typography, illustrations) — creativity in UI/UX is encouraged and rewarded. There is no Figma file; designing the experience is part of the assignment.

User journey



3. Functional Requirements

Items marked **[Must]** are mandatory. Items marked **[Bonus]** are optional and earn extra credit only when the mandatory scope is solid.

3.1 Splash & Authentication

- **[Must]** Splash screen that checks for a stored session and routes to Home or Login.
- **[Must]** Login screen with username & password fields, inline validation (empty fields, min length), password visibility toggle, and a loading state on the submit button.
- **[Must]** Call `POST /auth/login`; store the access & refresh tokens securely (e.g. `flutter_secure_storage`).
- **[Must]** Show a clear message for invalid credentials (API returns HTTP 400 “Invalid credentials”).
- **[Bonus]** Automatically refresh an expired access token via `POST /auth/refresh` using an HTTP interceptor, then retry the original request.

3.2 Home — Product List

- **[Must]** Display products in a responsive grid/list showing thumbnail, title, price, discount and rating.
- **[Must]** Pagination with infinite scroll using `limit` & `skip` (page size 20); show a footer loader and stop when all items are loaded.
- **[Must]** Pull-to-refresh.
- **[Must]** Horizontal category chips loaded from the API; selecting one filters the list.
- **[Must]** Search bar with debounce (approx. 400 ms) using the search endpoint.
- **[Must]** Sort options: price low-to-high / high-to-low, rating, title.
- **[Must]** Skeleton / shimmer placeholders during first load; dedicated empty state for “no results”.

3.3 Product Details

- **[Must]** Image carousel with page indicator, title, brand (absent on some products, so handle null), price with discount, rating, stock status, description, and the list of customer reviews.
- **[Must]** “Add to cart” with quantity selector; disable it when the product is out of stock.
- **[Must]** Handle a non-existent product gracefully (e.g. `/products/9999` returns 404).
- **[Bonus]** Hero animation from list thumbnail to details image.

3.4 Cart & Checkout

- **[Must]** Add, remove and update item quantity; show subtotal, discount and grand total.
- **[Must]** Persist the cart locally so it survives an app restart (Hive, Isar, sqflite or shared_preferences).
- **[Must]** Cart badge on the navigation bar / app bar reflecting item count in real time.
- **[Must]** “Place order” calls `POST /carts/add` and shows a success screen; clear the cart on success. (DummyJSON simulates this — nothing is actually stored server-side.)

3.5 Profile

- **[Must]** Fetch the signed-in user via `GET /auth/me` and show avatar, name, email, phone.
- **[Must]** Logout that clears tokens and local session, then returns to Login.
- **[Must]** If the API returns 401 (expired token), or token refresh fails with 403, sign the user out with an explanatory message.

3.6 Bonus features (optional)

- Light / dark theme toggle that persists. · Wishlist / favourites stored locally.
- Offline mode: cache the last fetched product list and show a “You are offline” banner.
- Widget tests or a golden test. · Localization (e.g. English + Hindi).
- CI with GitHub Actions running `flutter analyze` and `flutter test`.

4. API Reference (Verified Endpoints)

Base URL: **`https://dummyjson.com`**. No API key is required. Every endpoint below was tested and returned the expected response on 16 October 2026.

> **Test credentials** — username: `emilys` password: `emilyspass`
> Any user from `GET /users` can also sign in with their listed username and password.

Method	Endpoint	Purpose	Status
POST	/auth/login body: {"username", "password", "expiresInMins": 30}	Sign in; returns accessToken, refreshToken & user	200 / 400
GET	/auth/me header: Authorization: Bearer <accessToken>	Current user profile	200 / 401
POST	/auth/refresh body: {"refreshToken", "expiresInMins": 30}	Issue a new access token	200 / 403
GET	/products?limit=20&skip=0	Paginated product list (response includes total, skip, limit)	200
GET	/products?sortBy=price&order=asc	Sorted list (order = asc | desc)	200
GET	/products?select=title,price,thumbnail	Return only selected fields (lighter payload)	200
GET	/products/{id}	Single product with images and reviews	200 / 404
GET	/products/search?q=phone	Full-text product search (supports limit/skip)	200
GET	/products/categories	Categories as {slug, name, url}	200
GET	/products/category-list	Category slugs as a plain string array	200
GET	/products/category/{slug}	Products in a category, e.g. smartphones	200
POST	/carts/add body: {"userId": 1, "products": [{"id": 144, "quantity": 2}]}	Simulated order / cart creation with computed totals	201
GET	/carts/user/{userId}	Existing carts of a user	200

Endpoints for testing your error handling

Use these during development to prove that your app handles failures gracefully. Please demonstrate at least two of them in your screen recording.

Scenario	How to simulate	Expected app behaviour
Server error	GET /http/500	Friendly error view with a Retry button; no crash, no raw stack trace.
Not found	GET /http/404 or GET /products/9999	“Product not found” state with a way back.
Slow network	GET /products?delay=3000 (delay in ms, max 5000)	Loader stays visible; request timeout handled.
Unauthorised	GET /auth/me with an invalid / expired token	Token refresh (bonus) or forced logout with message.

Scenario	How to simulate	Expected app behaviour
Bad refresh token	POST /auth/refresh with an invalid refreshToken (returns 403)	Forced logout with a “session expired” message.
Invalid login	POST /auth/login with a wrong password	Inline message: “Invalid username or password”.
No internet	Turn on airplane mode	Offline message and retry; cached data if bonus implemented.

5. Technical Requirements

5.1 Platform & tooling

- Latest **stable** Flutter & Dart SDK. Mention exact versions in the README (`flutter --version`).
- Null-safe, lint-clean code: `flutter analyze` must report **zero issues**. Use `flutter_lints` or `very_good_analysis`.
- Format code with `dart format`. No commented-out dead code, no hard-coded secrets.

5.2 Clean architecture

- Organise code **feature-first**, each feature split into **presentation**, **domain** and **data** layers.
- Domain layer is pure Dart: entities, repository *interfaces* and use cases, with no Flutter or HTTP imports.
- Data layer: DTO/models with `fromJson`/`toJson` (freezed / json_serializable welcome), remote & local data sources, repository implementations.
- Use dependency injection (get_it, injectable or Riverpod providers). Widgets never instantiate repositories.

5.3 State management

- Pick **one** of: **flutter_bloc (BLoC/Cubit)**, **Riverpod** or **Provider**, and use it consistently.
- Model UI states explicitly (e.g. initial / loading / success / empty / failure / loading-more).
- Explain your choice and its trade-offs in the README in 3–5 lines.

5.4 Networking & error handling

- Single configured HTTP client (`dio` or `http`) with base URL, connect/receive timeouts and a logging interceptor in debug builds only.
- Auth interceptor that attaches the Bearer token to protected requests.
- Map low-level exceptions (SocketException, timeouts, 4xx, 5xx, parse errors) to typed domain **Failures**; return them via `Either` / `Result` or sealed classes.
- Every network-driven screen must have loading, success, empty and error UI with a retry action.

5.5 Responsive layouts

- Use `LayoutBuilder` / `MediaQuery` breakpoints. Minimum expectations:
  - **Compact** (< 600 dp): 2-column grid, bottom navigation bar.
  - **Medium** (600–1024 dp): 3–4 column grid, navigation rail.
  - **Expanded** (> 1024 dp / tablet landscape): master–detail list + details side by side.
- No overflow errors (yellow-black stripes) in any orientation or with system font scaled to 1.3×.

5.6 UI / UX expectations

- Material 3 with a centralised `ThemeData` (colour scheme, text theme, component themes).
- Consistent spacing scale, reusable widgets (buttons, cards, error view, empty view).
- Cached network images with placeholders (e.g. `cached_network_image`).
- Accessible tap targets (min 48 dp), sufficient contrast, semantic labels on icon buttons.
- Subtle, purposeful animations; avoid jank (use const constructors, avoid unnecessary rebuilds).

5.7 Unit testing

- **Minimum 15 meaningful unit tests**, covering at least:
  - Repository implementations: success, server failure and network failure paths.
  - Use cases (e.g. login, get products, search, cart total calculation).
  - State classes (Bloc/Cubit/Notifier) emitting the correct sequence of states (e.g. with `bloc_test`).
  - JSON model parsing using fixture files.
- Use `mocktail` or `mockito`. All tests must pass with `flutter test`; include a coverage summary in the README (bonus: ≥ 60%).

5.8 Suggested folder structure (reference only)

```
lib/
  core/
    di/                 # service locator / providers
    network/            # dio client, interceptors, api endpoints
    error/              # exceptions.dart, failures.dart
    theme/              # colours, typography, ThemeData
    utils/              # debouncer, formatters, breakpoints
    widgets/            # shared UI: ErrorView, EmptyView, Shimmer...
  features/
    auth/
      data/             # models, datasources, repositories_impl
      domain/           # entities, repositories, usecases
      presentation/     # bloc|providers, pages, widgets
    products/  ...
    cart/      ...
    profile/   ...
  app.dart
  main.dart
test/
  fixtures/             # sample JSON responses
  features/...          # mirrors lib/ structure
```

6. Git & README Guidelines

6.1 Git commits

- Initialise the repository at the **start** of the assignment and commit as you go. **A single "initial commit" containing the full project will be heavily penalised.**
- Aim for **20+ small, atomic commits**, each doing one logical thing.
- Follow **Conventional Commits**: `feat:`, `fix:`, `refactor:`, `test:`, `docs:`, `chore:`, `style:`.
- Optional but appreciated: feature branches merged into `main` via pull requests.
- Add a proper `.gitignore` (no `build/`, `.dart_tool/`, keystores or IDE files committed).

```
feat(auth): add login screen with form validation
feat(products): implement paginated product list with infinite scroll
fix(cart): prevent negative quantity on decrement
test(products): add unit tests for ProductRepositoryImpl failure paths
refactor(core): extract ErrorView into shared widgets
docs(readme): add architecture diagram and setup steps
```

6.2 README.md — required sections

The README is the first thing our reviewers read. Treat it like documentation for a teammate.

#	Section	Contents
1	Title & overview	App name, one-paragraph summary, link to APK and demo video.
2	Screenshots	At least 4 phone screenshots and 2 tablet/landscape screenshots (or a GIF).
3	Features	Checklist of implemented Must and Bonus features; clearly list anything not done.
4	Tech stack	Flutter/Dart versions and key packages with one line on why each was chosen.
5	Architecture	Layer diagram + folder structure + data flow of one request (UI → API → UI).
6	State management	Chosen solution and justification.
7	Error handling	How exceptions are mapped to failures and surfaced to the user.
8	Getting started	Prerequisites, clone, `flutter pub get`, code generation, run, build APK.
9	Testing	Command to run tests, number of tests, coverage summary.
10	Test credentials	emilys / emilyspass.
11	Assumptions & trade-offs	Decisions you made and what you would improve with more time.
12	Author	Your name, email, LinkedIn / portfolio.

7. Deliverables & Submission

Submit **all three** of the following by **21 October 2026, 11:59 PM IST**:

#	Deliverable	Requirements
1	**GitHub repository link**	Public repository (or private with access granted to the reviewer we specify). Name it `prosessed-flutter-assignment`. Must contain the full source, tests and README. Do not push commits after the deadline; the last commit timestamp will be checked.
2	**Google Drive link**	One folder named `Prosessed_Flutter_<FullName>`, shared as **"Anyone with the link – Viewer"**, containing: • the release APK • a 2–4 minute screen recording walking through all features, including at least two error scenarios and the tablet/landscape layout • a `screenshots/` folder
3	**Android APK**	Release build: `flutter build apk --release`. Name it `ShopSphere_<FullName>_v1.0.0.apk`. Must install and run on Android 8.0 (API 26)+. Upload to Google Drive **and** attach it to a GitHub Release tagged `v1.0.0`.

How to submit

Reply to the email in which you received this assignment, using the subject line and template below.

```
Subject: Flutter Intern Assignment Submission - <Full Name>

Full name        :
Email / Phone    :
GitHub repository: https://github.com/<username>/prosessed-flutter-assignment
Google Drive link: https://drive.google.com/drive/folders/<folder-id>
APK release link : https://github.com/<username>/prosessed-flutter-assignment/releases
State management : BLoC / Riverpod / Provider
Time spent       : ~ hours
Notes            : (anything we should know)
```

Important: Before sending, open every link in an incognito/private window to confirm it is accessible without signing in. Inaccessible links, broken APKs or empty repositories cannot be evaluated.

8. Evaluation Rubric

Submissions are scored out of **100**. Bonus features can add up to **10** extra points but cannot compensate for missing mandatory requirements.

Criterion	Weight	Excellent looks like
UI / UX design & polish	20	Cohesive visual design, delightful details, all states designed, accessible.
Clean architecture & code quality	15	Clear layers, SOLID, DI, small focused classes, readable naming.
State management	15	Predictable states, no logic in widgets, efficient rebuilds.
API integration & error handling	15	Typed models, robust interceptor, every failure mapped and recoverable.

Criterion	Weight	Excellent looks like
Unit testing	10	15+ meaningful passing tests across layers; good use of mocks and fixtures.
Responsive layouts	10	Phone, tablet, portrait and landscape all feel intentional; zero overflows.
Git commit history	5	Incremental, conventional, descriptive commits that show progression.
README & documentation	5	All required sections; a reviewer can run the project in under 5 minutes.
Deliverables & functionality	5	APK installs and works; links accessible; demo video clear.
Total	**100**	**+ up to 10 bonus points**

Automatic disqualification

- Plagiarised code or a submission copied from another candidate or public repository.
- APK that does not install or crashes on launch.
- Submission received after the deadline without prior approval.

9. Rules, Policies & FAQ

- **Original work:** this is an individual assignment. You may use documentation, Stack Overflow and open-source packages; credit any significant snippet you adapt.
- **AI tools:** AI assistants are permitted, but you must fully understand every line you submit. Shortlisted candidates will walk us through their code and make a small live change in a follow-up technical interview. Please mention in the README how you used AI tools, if at all.
- **Packages:** use any pub.dev packages you like, but be ready to justify each one.
- **Partial submissions:** if you cannot finish everything, submit what you have and clearly document what is missing and how you would approach it. Honesty is valued.
- **Extensions:** request any extension *before* the deadline by replying to the assignment email with a reason.
- **Confidentiality:** please do not share this document publicly. Your repository may be public.

FAQ

Question	Answer
Can I use a different public API?	No. Please use DummyJSON so all submissions are compared fairly.
Do I need an iOS build?	No. Android APK only. Your code should still compile for iOS without changes.
Is the API data persisted?	No. DummyJSON simulates POST/PUT/DELETE; responses are realistic but nothing is saved. That is why the cart must be persisted locally.
What if DummyJSON is down?	Rare, but if it happens, document it in your submission notes. Your error states will be shown to good effect!
Who do I contact with questions?	Reply to the assignment email. We aim to respond within 24 hours on working days.

10. Pre-Submission Checklist

- All **[Must]** features implemented and manually tested on a real device or emulator.
- `flutter analyze` reports zero issues; code formatted with `dart format`.
- `flutter test` passes with at least 15 unit tests.
- Checked on a phone and a tablet emulator, in portrait and landscape, with large font scale.
- Error states verified using /http/500, /products/9999, ?delay=3000 and airplane mode.
- Git history has 20+ meaningful Conventional Commits; no build artefacts or secrets committed.
- README contains all 12 required sections, screenshots and test credentials.
- Release APK built, installed and smoke-tested; attached to GitHub Release v1.0.0.
- Google Drive folder contains APK, demo video and screenshots; shared as “Anyone with the link – Viewer”.
- All links opened successfully in an incognito window.
- Submission email sent before **21 October 2026, 11:59 PM IST**.

> **We look forward to seeing what you build!**
> Good luck from the Prosessed Engineering Team. Remember: clarity, quality and honesty matter more than quantity.